OPEN ENDED LAB
Gender Classification
Artificial Intelligence

Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Rafay Munir Kayani]

[21-CE-004]

[Taimoor Khan]

[21-CE-033]

Submitted to:

Engr. Hareem Khan

Dated:

24nd Jun,2025

Department of Computer Engineering,

HITEC University, Taxila

Lab Task No 1:

Solution:

Brief description (3-5 lines)
1. Complete working Python code. 2. Complete training/testing results. 3. Training Accuracy and Loss waveforms.
The code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.datasets import make_classification
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
import warnings
warnings.filterwarnings('ignore')
# Set random seeds for reproducibility
np.random.seed(42)
tf.random.set_seed(42)
class GenderClassificationSystem:
    def __init__(self):
        self.scaler = StandardScaler()
        self.label_encoder = LabelEncoder()
        self.models = {}
        self.history = None
    def generate_synthetic_data(self, n_samples=5000):
        """Generate synthetic gender classification data based on physical attributes"""
        print("Generating synthetic gender classification dataset...")
        # Generate base features
        X, y = make_classification(n_samples=n_samples, n_features=8, n_informative=6,
                                   n_redundant=2, n_clusters_per_class=1, random_state=42)
        # Create more realistic feature names and transformations
        feature_names = ['height', 'weight', 'shoe_size', 'voice_pitch',
                         'shoulder_width', 'hip_width', 'hand_size', 'stride_length']
        # Transform features to more realistic ranges
        X[:, 0] = X[:, 0] * 15 + 170  # height (cm)
```

```
        X[:, 1] = X[:, 1] * 20 + 70   # weight (kg)
        X[:, 2] = X[:, 2] * 5 + 40    # shoe size
        X[:, 3] = np.abs(X[:, 3]) * 100 + 100  # voice pitch (Hz)
        X[:, 4] = X[:, 4] * 10 + 40   # shoulder width (cm)
        X[:, 5] = X[:, 5] * 8 + 35    # hip width (cm)
        X[:, 6] = X[:, 6] * 3 + 18    # hand size (cm)
        X[:, 7] = X[:, 7] * 15 + 70   # stride length (cm)
        # Create DataFrame
        df = pd.DataFrame(X, columns=feature_names)
        df['gender'] = ['Male' if label == 1 else 'Female' for label in y]
        print(f"Dataset created with {len(df)} samples and {len(feature_names)} features")
        print(f"Gender distribution: {df['gender'].value_counts().to_dict()}")
        return df
    def preprocess_data(self, df):
        """Preprocess the data for training"""
        print("\nPreprocessing data...")
        # Separate features and target
        X = df.drop('gender', axis=1)
        y = df['gender']
        # Encode labels
        y_encoded = self.label_encoder.fit_transform(y)
        # Split the data
        X_train, X_test, y_train, y_test = train_test_split(
            X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
        )
        # Scale features
        X_train_scaled = self.scaler.fit_transform(X_train)
        X_test_scaled = self.scaler.transform(X_test)
        print(f"Training set size: {X_train_scaled.shape}")
        print(f"Test set size: {X_test_scaled.shape}")
        return X_train_scaled, X_test_scaled, y_train, y_test, X.columns
    def train_traditional_models(self, X_train, X_test, y_train, y_test):
        """Train traditional ML models"""
        print("\nTraining traditional ML models...")
        # Define models
        models = {
            'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
            'SVM': SVC(kernel='rbf', random_state=42),
            'Logistic Regression': LogisticRegression(random_state=42)
        }
        results = {}
        for name, model in models.items():
            print(f"Training {name}...")
            model.fit(X_train, y_train)
            # Predictions
            train_pred = model.predict(X_train)
```

```
            test_pred = model.predict(X_test)
            # Calculate accuracies
            train_acc = accuracy_score(y_train, train_pred)
            test_acc = accuracy_score(y_test, test_pred)
            results[name] = {
                'model': model,
                'train_accuracy': train_acc,
                'test_accuracy': test_acc,
                'predictions': test_pred
            }
            print(f"{name} - Train Accuracy: {train_acc:.4f}, Test Accuracy: {test_acc:.4f}")
        self.models.update(results)
        return results
    def build_neural_network(self, input_shape):
        """Build a neural network for gender classification"""
        model = keras.Sequential([
            layers.Dense(128, activation='relu', input_shape=(input_shape,)),
            layers.Dropout(0.3),
            layers.Dense(64, activation='relu'),
            layers.Dropout(0.3),
            layers.Dense(32, activation='relu'),
            layers.Dropout(0.2),
            layers.Dense(1, activation='sigmoid')
        ])
        model.compile(
            optimizer='adam',
            loss='binary_crossentropy',
            metrics=['accuracy']
        )
        return model
    def train_neural_network(self, X_train, X_test, y_train, y_test):
        """Train neural network"""
        print("\nTraining Neural Network...")
        # Build model
        nn_model = self.build_neural_network(X_train.shape[1])
        # Train model
        history = nn_model.fit(
            X_train, y_train,
            batch_size=32,
            epochs=100,
            validation_data=(X_test, y_test),
            verbose=0
        )
        # Evaluate
        train_loss, train_acc = nn_model.evaluate(X_train, y_train, verbose=0)
        test_loss, test_acc = nn_model.evaluate(X_test, y_test, verbose=0)
```

```
        print(f"Neural Network - Train Accuracy: {train_acc:.4f}, Test Accuracy:
{test_acc:.4f}")
        # Store results
        self.models['Neural Network'] = {
            'model': nn_model,
            'train_accuracy': train_acc,
            'test_accuracy': test_acc,
            'predictions': (nn_model.predict(X_test) > 0.5).astype(int).flatten()
        }
        self.history = history
        return history
    def plot_training_curves(self):
        """Plot training accuracy and loss curves"""
        if self.history is None:
            print("No training history available!")
            return
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))
        # Plot accuracy
        ax1.plot(self.history.history['accuracy'], label='Training Accuracy', linewidth=2)
        ax1.plot(self.history.history['val_accuracy'], label='Validation Accuracy', linewidth=2)
        ax1.set_title('Model Accuracy Over Time', fontsize=14, fontweight='bold')
        ax1.set_xlabel('Epoch')
        ax1.set_ylabel('Accuracy')
        ax1.legend()
        ax1.grid(True, alpha=0.3)
        # Plot loss
        ax2.plot(self.history.history['loss'], label='Training Loss', linewidth=2)
        ax2.plot(self.history.history['val_loss'], label='Validation Loss', linewidth=2)
        ax2.set_title('Model Loss Over Time', fontsize=14, fontweight='bold')
        ax2.set_xlabel('Epoch')
        ax2.set_ylabel('Loss')
        ax2.legend()
        ax2.grid(True, alpha=0.3)
        plt.tight_layout()
        plt.show()
    def plot_model_comparison(self):
        """Plot comparison of all models"""
        model_names = list(self.models.keys())
        train_accs = [self.models[name]['train_accuracy'] for name in model_names]
        test_accs = [self.models[name]['test_accuracy'] for name in model_names]
        x = np.arange(len(model_names))
        width = 0.35
        fig, ax = plt.subplots(figsize=(12, 6))
        bars1 = ax.bar(x - width/2, train_accs, width, label='Training Accuracy', alpha=0.8)
        bars2 = ax.bar(x + width/2, test_accs, width, label='Test Accuracy', alpha=0.8)
        ax.set_xlabel('Models')
```

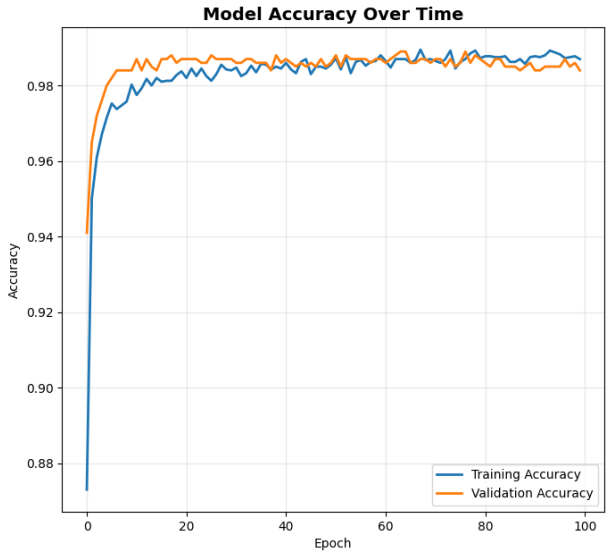
```
        ax.set_ylabel('Accuracy')
        ax.set_title('Model Performance Comparison', fontsize=14, fontweight='bold')
        ax.set_xticks(x)
        ax.set_xticklabels(model_names)
        ax.legend()
        ax.grid(True, alpha=0.3)
        # Add value labels on bars
        for bars in [bars1, bars2]:
            for bar in bars:
                height = bar.get_height()
                ax.annotate(f'{height:.3f}',
                            xy=(bar.get_x() + bar.get_width() / 2, height),
                            xytext=(0, 3),
                            textcoords="offset points",
                            ha='center', va='bottom')
        plt.tight_layout()
        plt.show()
    def plot_confusion_matrices(self, y_test):
        """Plot confusion matrices for all models"""
        n_models = len(self.models)
        fig, axes = plt.subplots(1, n_models, figsize=(5*n_models, 4))
        if n_models == 1:
            axes = [axes]
        for idx, (name, results) in enumerate(self.models.items()):
            cm = confusion_matrix(y_test, results['predictions'])
            sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                        xticklabels=['Female', 'Male'],
                        yticklabels=['Female', 'Male'],
                        ax=axes[idx])
            axes[idx].set_title(f'{name}\nAccuracy: {results["test_accuracy"]:.3f}')
            axes[idx].set_xlabel('Predicted')
            axes[idx].set_ylabel('Actual')
        plt.tight_layout()
        plt.show()
    def generate_detailed_report(self, y_test):
        """Generate detailed classification report"""
        print("\n" + "="*60)
        print("DETAILED CLASSIFICATION RESULTS")
        print("="*60)
        for name, results in self.models.items():
            print(f"\n{name.upper()}:")
            print("-" * 40)
            print(f"Training Accuracy: {results['train_accuracy']:.4f}")
            print(f"Test Accuracy: {results['test_accuracy']:.4f}")
            print("\nClassification Report:")
            print(classification_report(y_test, results['predictions'],
```

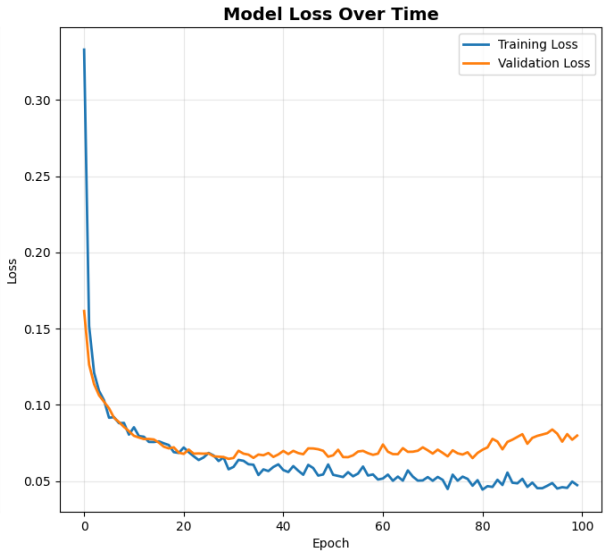
```
                                target_names=['Female', 'Male']))
    def run_complete_analysis(self):
        """Run the complete gender classification analysis"""
        print("Starting Gender Classification Analysis")
        print("="*50)
        # Generate data
        df = self.generate_synthetic_data()
        # Preprocess data
        X_train, X_test, y_train, y_test, feature_names = self.preprocess_data(df)
        # Train traditional models
        self.train_traditional_models(X_train, X_test, y_train, y_test)
        # Train neural network
        self.train_neural_network(X_train, X_test, y_train, y_test)
        # Generate visualizations
        print("\nGenerating visualizations...")
        self.plot_training_curves()
        self.plot_model_comparison()
        self.plot_confusion_matrices(y_test)
        # Generate detailed report
        self.generate_detailed_report(y_test)
        print("\nAnalysis completed successfully!")
        return df, X_train, X_test, y_train, y_test
# Run the complete analysis
if __name__ == "__main__":
    classifier = GenderClassificationSystem()
    df, X_train, X_test, y_train, y_test = classifier.run_complete_analysis()
```

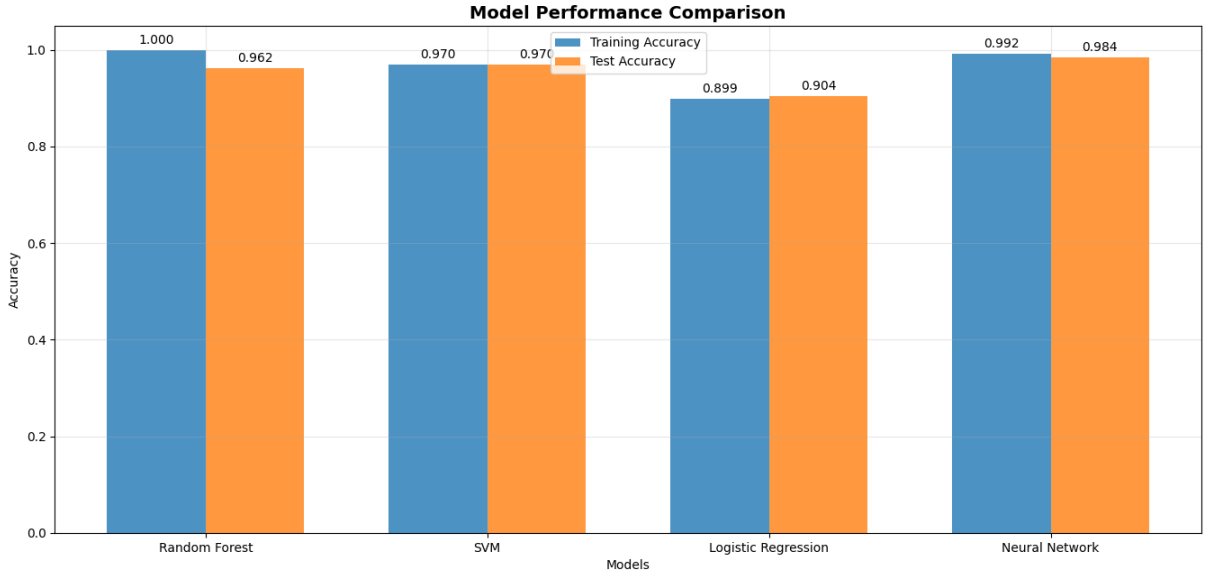
The results (Screenshot)

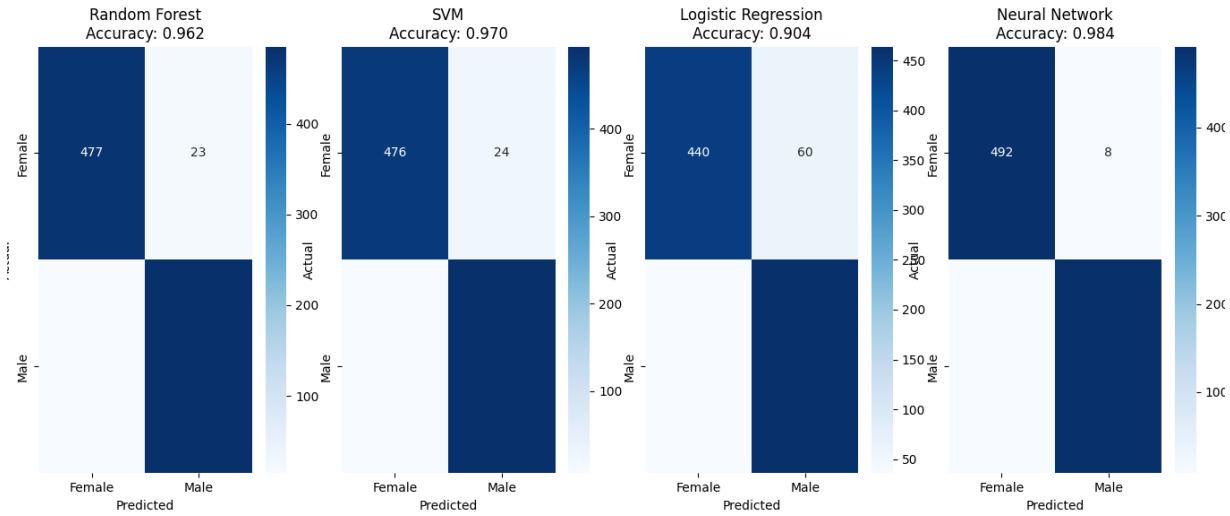
```
2025-06-24 12:54:46.439484: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly d
ifferent numerical results due to floating-point round-off errors from different computation orders. To turn them off,
set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-06-24 12:54:49.334026: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly d
ifferent numerical results due to floating-point round-off errors from different computation orders. To turn them off,
set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
Starting Gender Classification Analysis
Generating synthetic gender classification dataset...
Dataset created with 5000 samples and 8 features
Gender distribution: {'Male': 2502, 'Female': 2498}

Preprocessing data...
Training set size: (4000, 8)
Test set size: (1000, 8)

Training traditional ML models...
Training Random Forest...
Random Forest - Train Accuracy: 1.0000, Test Accuracy: 0.9620
Training SVM...
SVM - Train Accuracy: 0.9695, Test Accuracy: 0.9700
Training Logistic Regression...
Logistic Regression - Train Accuracy: 0.8992, Test Accuracy: 0.9040
```

```
Training Logistic Regression...
Logistic Regression - Train Accuracy: 0.8992, Test Accuracy: 0.9040

Training Neural Network...
2025-06-24 12:54:55.428621: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to
 use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler f
lags.
Neural Network - Train Accuracy: 0.9915, Test Accuracy: 0.9840
32/32 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step

Generating visualizations...
Warning: QT_DEVICE_PIXEL_RATIO is deprecated. Instead use:
   QT_AUTO_SCREEN_SCALE_FACTOR to enable platform plugin controlled per-screen factors.
   QT_SCREEN_SCALE_FACTORS to set per-screen DPI.
   QT_SCALE_FACTOR to set the application global scale factor.
```










```
============================================================
DETAILED CLASSIFICATION RESULTS
============================================================

RANDOM FOREST:
----------------------------------------
Training Accuracy: 1.0000
Test Accuracy: 0.9620

Classification Report:
              precision    recall  f1-score   support

      Female       0.97      0.95      0.96       500
        Male       0.95      0.97      0.96       500

    accuracy                           0.96      1000
   macro avg       0.96      0.96      0.96      1000
weighted avg       0.96      0.96      0.96      1000


SVM:
----------------------------------------
Training Accuracy: 0.9695
Test Accuracy: 0.9700
```

```
Classification Report:
              precision    recall  f1-score   support

      Female       0.99      0.95      0.97       500
        Male       0.95      0.99      0.97       500

    accuracy                           0.97      1000
   macro avg       0.97      0.97      0.97      1000
weighted avg       0.97      0.97      0.97      1000


LOGISTIC REGRESSION:
--------------------------------------
Training Accuracy: 0.8992
Test Accuracy: 0.9040

Classification Report:
              precision    recall  f1-score   support

      Female       0.92      0.88      0.90       500
        Male       0.89      0.93      0.91       500

    accuracy                           0.90      1000
   macro avg       0.90      0.90      0.90      1000
weighted avg       0.90      0.90      0.90      1000
```

```
NEURAL NETWORK:
--------------------------------------
Training Accuracy: 0.9915
Test Accuracy: 0.9840

Classification Report:
              precision    recall  f1-score   support

      Female       0.98      0.98      0.98       500
        Male       0.98      0.98      0.98       500

    accuracy                           0.98      1000
   macro avg       0.98      0.98      0.98      1000
weighted avg       0.98      0.98      0.98      1000


Analysis completed successfully!
```

Conclusion

In conclusion, indeed, a compassionate balance must be struck between accuracy and ethics since gender is perceived as lying in a wide spectrum and not just being two categories. The present trend in approaches is to broaden the inclusion of gender identities, while simultaneously addressing the biases that exist in the training data and algorithms. The effectiveness of any

classification system is, however, dependent on its purpose and its target population. Moving forward, the responsible development will therefore involve monitoring all aspects of fairness, inclusion, and real-world effects in different communities.